

1 存储系统

1.1 存储器的性能指标

1. 存储容量 = 存储字数 × 存储字长

- 存储字数：表示存储器可编址的存储单元个数（即地址空间大小），决定所需地址位数
 - ① $4M$ 的存储字数：即一共有 $4M = 4 \times 2^{20} = 2^{22}$ 个存储单元，可使用引脚复用技术，分为行、列引脚，共 11 个引脚，先后两次输入，需要 11 根地址引脚
- 存储字长：表示一个存储单元包含的数据位数，即每个存储单元的比特数据，也表示数据引脚的数量
 - ① 按字节编址：存储容量 $4MB = 4M \times 8bit$ 。存储字长 $8bit$ ，即一个地址对应 $8bit$ 数据，8 根数据引脚
 - ② 按字编址：存储容量 $16MB = 4M \times 32bit$
 - ③ 按半字编址：存储容量 $8MB = 4M \times 16bit$

2. 单位成本：位价 = 总成本/总容量

3. 存储速度：数据传输速率，即每秒传送信息的位数。存储速度 = 存储字长/存取周期 = 存取时间 + 恢复时间

- 存取时间 (T_a)：从启动一次存储器操作到完成该操作所经历的时间
- 存取周期 (T_m)：存储器进行一次完整的读/写操作所需的全部时间，即连续两次独立访问存储器操作之间所需的最小时间间隔
- 主存带宽 (B_m , 数据传输速率)：表示每秒从主存进出信息的最大数量

1.2 RAM (Random Access Memory, 随机存储器)

RAM = SRAM + DRAM

1. SRAM (Static Random Access Memory, 静态随机存储器)：即使信息被读出后，它仍保持器原状态而不需要再生（非破坏性读出）
2. DRAM (Dynamic Random Access Memory, 动态随机存储器)：Cache。必须定时刷新和读后再生
3. 主存由 RAM 和 ROM 构成

1.2.1 DRAM

1. 刷新周期：对同一行进行相邻两次刷新的时间间隔，通常取 $2ms$ 。e.g. $64K \times 1bit$ 的 DRAM 芯片构成 $128K \times 8bit$ 的存储器，若采用异步刷新方式，每单元存储间隔不超过 $2ms$ ，则生成的刷新信号的间隔时间是？若采用集中刷新方式，则存储器刷新一遍最少用多少个读/写周期？

解析

$64K \times 1bit$ 由一个 256×256 的为平面组成
构成存储器的所有芯片同时按行刷新，每个芯片有 256 行，所以存储器所有单元刷新一遍至少需要 256 次刷新操作
若采用异步刷新方式，则相邻两次刷新信息的时间间隔为 $2ms/245$
若采用集中刷新方式，则整个存储器刷新一遍至少需要 256 个读/写周期

2. DRAM 的刷新单位是行，逐行定时刷新，没刷新一行，需消耗一个存取周期
3. 每个刷新周期需刷新 $2^{\text{行地址数}}$ ，即若行地址 6bit ，则每个刷新周期需刷新 $2^6 = 64$ 次，消耗 64 个存取周期
4. 再生仅需恢复被读出的那些单元的数据
5. 行缓冲器：缓存指定行中每列的数据，大小 = 列数 $\times$ 每列的数据位宽，常用 SRAM 实现，可支持突发传输
6. DRAM 存取周期 $>$ SRAM 存取周期 $\Rightarrow$ 一次突发传输决不能跨越 DRAM 行边界

1.3 多模块存储器

1. 单体多字存储器
2. 多体并行存储器
 - 高位交叉编址
 - ① 连续读取 m 个字所需的时间 $t = mT$
 - 低位交叉编址：程序连续存放在相邻模块中。可采用轮流启动或同时启动方式
 - ① 模块的存取周期为 T ，总线周期 r ，存储器交叉模块数应大于或等于 $m = \frac{T}{r}$
 - ② 连续存取 m 个字所需的时间 $t = T + (m - 1)r$

1.4 外部存储器

1.4.1 磁盘存储器

1. 磁盘有 100 个柱面，每个柱面有 8 个磁道 == 磁盘有 8 个盘面，每个盘面 100 个磁道
2. 磁盘地址结构：驱动器号 - 柱面（磁道）号 - 盘面（磁头）号 - 扇区号
3. 逻辑地址 = 柱面号 $\times$ 盘面数 $\times$ 扇区数 + 盘面号 $\times$ 扇区数 + 扇区号

1.4.2 磁盘的管理

1. 低级格式化（物理格式化）：将磁盘分成扇区（扇区由头部、数据区域和尾部组成，头部和尾部包含了一些磁盘控制器的使用信息），以便磁盘控制器能够进行读/写操作
2. 磁盘分区：每个分区由一个或多个柱面组成，每个分区的起始扇区和大小都记录在磁盘主引导记录的分区表中
3. 高级格式化（逻辑格式化）：将初始的文件系统数据结构存储到磁盘上
4. 块：操作系统将多个相邻的扇区组合在一起，形成块
5. 引导块（Boot Block）是磁盘最前面的一小块区域，用来存放启动计算机所需的引导程序（Boot Loader）

1.4.3 磁盘的性能指标

1. 记录密度：盘片单位面积上记录的二进制数据量

- 道密度：沿磁盘半径方向单位长度上的磁道数
- 位密度：磁道单位长度上能记录的二进制代码位数
- 面密度：位密度和道密度的乘积

2. 磁盘的容量

- 非格式化容量：磁记录编码可利用的磁化单位总数。非格式化容量 = 记录面数 × 柱面数 × 每条磁道的磁化单位数
- 格式化容量：按照某种特定的记录格式所能存储信息的总量。格式化容量 = 记录面数 × 柱面数 × 每道扇区数 × 每个扇区的容量
- 格式化后的容量比非格式化容量要小

3. ★存取时间 = 平均寻道时间 + 平均旋转延迟时间 + 传输时间

- 寻道时间：磁头移动到目的磁道的时间。包括启动磁头臂的时间 s 。寻道时间与磁盘调度算法密切相关。设 m 是跨越一个磁道所需的时间，则跨越 n 条磁道的寻道时间：

$$T_s = mn + s$$

- 平均寻道时间：从最外道移动到最内道时间的一半，和转速无关
- 旋转延迟时间：磁头定位到要读/写扇区的时间
- 平均旋转延迟时间：旋转半周的时间，和转速有关。设磁盘的旋转速度为 r ，则

$$T_r = \frac{1}{2r}$$

- 传输时间：传输数据所花费的时间。设读/写字节数 b 、磁盘的旋转速度 r 且一个磁道的字节数为 N ，则

$$T_t = \frac{1}{r} \times \frac{b}{N} = \frac{b}{rN}$$

4. 数据传输速率：在单位时间内向主机传送数据的字节数。假设磁盘转数为 r 转/秒，每条磁道容量为 N 字节，则数据传输速率为

$$D_r = rN = \frac{\text{每个磁道容量}}{\text{读一个磁道数据的时间}}$$

5. 读一个磁道数据的时间 = 磁盘旋转一周的时间 - 通过扇区间隙的总时间

1.4.4 磁盘调度算法

1. 先来先服务 (First Come First Served, FCFS) 算法

2. 最短寻道时间优先 (Shortest Seek Time First, SSTF) 算法：每次选择调度的是例当前最近的磁道，使每次的寻道时间最短。会产生“饥饿”现象

3. 扫描 (SCAN) 算法 (电梯调度算法)：只有磁头移动到最外侧磁道时才向内移动，移动到最内侧磁道时才能向外移动

- 偏向于处理那些接近最里或最外的磁道的访问请求
- LOOK 调度：磁头只需移动到最远端的一个请求即可返回，不需要达到磁盘端点

4. 循环扫描 (Circular SCAN, C-SCAN) 算法：在 SCAN 的基础上规定磁头单向移动来提供服务，返回时直接快速移动至起始端而不服务任何请求

- C-LOOK 调度：磁头只需移动到最远端的一个请求即可返回，不需要达到磁盘端点

1.4.5 减少延迟时间的方法

1. 磁盘是连续自转设备，磁头读入一个扇区后，需要经过短暂的处理时间，才能开始读入下一个扇区
2. 柱面切换代价（需移动磁头）> 盘面切换代价
3. 交替编号：让逻辑上相邻的块在物理上保持一定的间隔，则读入多个连续块是能够减少延迟时间
4. 错位命名：不同盘面上的相邻扇区编号错开，避免盘面切换时错过目标扇区，提高连续读取速度

1.4.6 提高文件访问速度方法

1. 改进文件的目录结构和检索目录的方法，以减少对目录的查找时间
2. 选取好的文件存储结构，以提高对文件的访问速度
3. 提高磁盘 I/O 速度，已实现文件中的数据在磁盘和内存之间的快速传送
 - 采用磁盘高速缓存
 - 调整磁盘请求顺序
 - 提前度
 - 延迟写
 - 优化物理块的分布
 - 虚拟盘
 - 采用磁盘阵列 RAID

1.4.7 固态硬盘

1. 数据以页为单位读/写，以块为单位擦除
2. 每个擦除块（Erase Block）只能擦除有限次数
3. 磨损均衡
 - 动态磨损均衡：写入数据时，优先选择擦除次数少的新闪存块
 - 静态磨损均衡：连那些长期不变的数据也会被主动迁移，使原本几乎没有擦写的块重新参与使用，从而让整个 SSD 的擦写次数更加均匀，延长使用寿命

1.5 高速缓冲存储器

1. 时间局部性：某个数据/指令在短时间内被反复访问
2. 空间局部性：最近的未来要用到的信息，很可能与现在正在使用的信息在存储空间上是临近的
3. CPU 与 Cache 之间的数据交换以字（机器字长）为单位
4. Cache 与主存之间的数据交换以 Cache 块为单位。Cache 块也称 Cache 行，每块由若干字节组成，块的长度称为块长，也称行长
5. Cache 命中率：CPU 与访问的信息已在 Cache 中的比率
6. Cache-主存系统的平均访问时间

- t_c : 命中时的 Cache 访问时间
- t_m : 未命中 (缺失) 时的访问时间
- H : 命中率
- $1 - H$: 未命中率 (缺失率)
- 串行访问: $T_a = Ht_c + (1 - H)(t_c + t_m)$
- 并行访问: $T_a = Ht_c + (1 - H)t_m$

1.5.1 Cache 与主存映射方式

1. 标记项: 包含有效位、脏位 (修改位)、替换算法位、标记位
2. 标记位: 指明 Cache 块是主存中哪一块的副本
3. 有效位: 说明 Cache 行中的信息是否有效
4. 地址映射: 把主存地址空间映射到 Cache 地址空间, 即把存放在主存中的信息按照某种规则装入 Cache

- 直接映射: 主存中的每一块只能装入 Cache 中的唯一位置

- ① **块冲突**: 若这个位置已有内容, 则产生**块冲突**, 原来的块将无条件地被替换出去
- ② 命中: 根据访存地址中间的 Cache 行号位数, 找到对应 Cache 行, 将对应 Cache 行中的标记与主存地址的标记位数 (高位) 进行比较, 若相等且有效位为 1, 则命中

- ③ Cache 不存 Cache 行号和块内地址

- ④ 计算

- (a) 主存容量 $\Rightarrow$ 主存地址位数: 主存容量 = 存储字数 $\times$ 存储字长, 主存容量 1MB, 按字节编址, 存储字数 = $1MB/1B = 2^{20}$, 即主存地址 20bit

- (b) Cache 有效容量: Cache 中存数据的容量, 即数据块总大小

- (c) Cache 行结构 = 有效位 + 脏位 (修改位) + 替换算法位 + Tag + 数据位 (块大小 32B $\Rightarrow$ 数据位 $32 * 8\text{bit} = 256\text{bit}$)

- (d) Cache 容量 = (数据块 (行长) + Tag + 控制位) * 行数/块数: Cache 数据区大小 32KB, 主存块大小 32B, 行数 = $32KB/32B = 2^{10}$, 即 10bit 行号

- (e) 行长/块长 $\Rightarrow$ 块内地址位数: 每块 $16B = 2^4B$, 即块内地址 4bit

- (f) 行数/块数 $\Rightarrow$ 行号位数: Cache 行号位数 = $\log_2$ Cache 行数

- (g) 主存块号 = $\frac{\text{主存地址}}{\text{行长}}$

- (h) Cache 行号 = 主存块号 mod Cache 总行数

- (i) 主存地址结构: 标记位 - Cache 行号 - 块内地址 (与块长、行长有关)

- (j) 主存块号 = 标记位 + Cache 行号

- 全相联映射: 主存中的每一块可以装入 Cache 中的任何位置

- ① 优点

- (a) Cache 块的冲突概率低, 只要有空闲 Cache 行, 就不会发生冲突

- (b) 空间利用率高
- (c) 命中率高
- ② 缺点
 - (a) 标记的比较速度较慢
 - (b) 实现成本较高，通常需采用按内容寻址的相联存储器
- ③ 主存地址结构：标记 - 块内地址（与块长、行长有关）
- 组相联映射：将 Cache 分成 Q 个大小相等的组，每个主存块可以固定组中的任意一行，即组间采用直接映射，组内采用全相联映射
 - ① 每组 Cache 行的数量越大，发生块冲突的概率越低
 - ② r 路组相联：每组中有 r 个 Cache 行
 - ③
$$\text{Cache 组数} = \frac{\text{Cache 总容量}}{\text{Cache 块大小} \times r}$$
 - ④ $\text{Cache 组号} = \text{主存块号} \bmod \text{Cache 组数} (Q)$
 - ⑤ 主存地址结构：标记 - 组号 - 块内地址（与块长、行长有关）
 - ⑥ $\text{主存块号} = \text{标记位} + \text{组号}$
 - ⑦ 命中：根据访存地址中间的组号找到对应的 Cache 组，将对应的 Cache 组中每个行的标记与主存地址的高位标记进行比较，若相等且有效位 1，则命中
- 计算

1.5.2 Cache 一致性问题

1. Cache 写操作命中

- 全写法（直写法，Write Through）：当 CPU 对 Cache 写命中时，把数据同时写入 Cache 和主存
 - ① 写缓冲（Write Buffer）：为减少全写法直接写入主存的时间消耗，在 Cache 和主存之间加一个写缓冲
 - ② CPU 同时写数据到 Cache 和写缓冲中，写缓冲再将内容写入主存
 - ③ 写缓冲是一个 FIFO 队列，可以解决速度不匹配的问题，但若出现频繁写时，会是写缓冲饱和溢出
- 回写法（写回法，Write Back）：当 CPU 对 Cache 写命中时，只把数据写入 Cache，只有当此块被替换出时才写回主存
 - ① 修改位（脏位）：为了减少写回主存的次数，若为 1，则说明对应的 Cache 行中的块被修改过，替换时须写回主存

2. Cache 写操作不命中

- 写分配法（Write Allocate）：把主存块调入 Cache，之后只修改 Cache，搭配 Write Back，标记 Dirty
- 非写分配法（Not-Write Allocate）：只更新主存单元，而不把主存块调入 Cache，搭配 Write Through

3. 现代计算机使用 Write Back + Write Allocate